COMMON WEB VULNERABILITIES
TryHackMe Learning & Practical Report

Prepared by: Kaustubh Ashish Bangar

Focus: Web Application Security / WAPT Foundations

Task: Task 4 — Common Web Vulnerabilities

Room Covered: IDOR - Santa's Little IDOR (Advent of Cyber 2025)

A comprehensive technical report documenting the methodologies, theoretical concepts, and practical exercises completed for the Authentication, Access Control, and IDOR phase of the IqraSec Academy WAPT Internship Phase 1.

1. Introduction

Access control vulnerabilities represent some of the most critical and prevalent security flaws in modern web applications. Specifically, Insecure Direct Object Reference (IDOR)—also more accurately described as an Authorization Bypass—occurs when an application provides direct access to objects based on user-supplied input without properly validating the user's permissions.

In this practical scenario, an investigation was conducted on the "TryPresentMe" website following reports of missing vouchers, targeted phishing emails containing non-public information, and the discovery of a suspicious account named "Sir Carrotbane". The objective was to assist the TBFC support staff in investigating the TryPresentMe infrastructure, identifying the root cause of the data leaks, and fixing the underlying vulnerabilities.

This report documents the theoretical principles of authentication and authorization, the mechanics of IDOR vulnerabilities, and the practical exploitation techniques used to uncover sensitive user data and perform horizontal privilege escalation.

2. Objective

The primary objective of this task is to understand and exploit access control flaws, specifically focusing on Insecure Direct Object References (IDOR).

Specific learning and practical objectives include:

1. **Identity Principles:** To thoroughly understand the fundamental differences between authentication (verifying identity) and authorization (verifying permissions).
2. **Privilege Escalation:** To differentiate between horizontal and vertical privilege escalation vectors.
3. **Vulnerability Identification:** To learn how to spot potential opportunities for IDORs in HTTP requests, parameters, and local storage.
4. **Exploitation Mechanisms:** To exploit an IDOR vulnerability to perform horizontal privilege escalation and access unauthorized user data.
5. **Obfuscation Analysis:** To identify how developers attempt to mask IDORs using encoding (Base64), hashing (MD5/SHA1), and deterministic generation (UUIDv1).

6. **Remediation:** To learn how to remediate these flaws by turning IDOR into Secure Direct Object Reference (SDOR) through robust server-side access controls.

3. Concept / Understanding

3.1 Authentication vs. Authorization

To comprehend the root cause of an IDOR vulnerability, one must strictly differentiate between authentication and authorization:

- **Authentication:** This is the process by which a system verifies *who* you are. It typically involves supplying credentials, such as a username and password. Once authenticated, the server issues session information (like a cookie or token), which is included in every subsequent request. This means authentication happens continuously with each request, verifying the active session.
- **Authorization:** This is the subsequent process where the web application verifies your *permissions*. It determines what actions you are allowed to perform or what data you are allowed to view (e.g., accessing an admin panel or viewing a specific banking account).

Crucially, authorization cannot occur before authentication; the application must know who the user is before it can check their permissions. IDOR vulnerabilities manifest when authentication is successful, but the subsequent authorization checks are either flawed or entirely absent.

3.2 Privilege Escalation

Exploiting access control flaws typically results in one of two forms of privilege escalation:

- **Vertical Privilege Escalation:** Occurs when an attacker gains access to features or functionalities that should be restricted to a higher-privileged role, such as a standard user executing administrative commands.
- **Horizontal Privilege Escalation:** Occurs when an attacker utilizes a feature they are authorized to use, but manipulates the request to gain access to data belonging to other users with the same privilege level. For example, viewing another customer's private account details. IDOR is most commonly a form of horizontal privilege escalation.

3.3 The Mechanics of IDOR (Authorization Bypass)

The term "Direct Object Reference" simply means the system uses an identifier (like `packageID=1001` or `/user/1`) to point to a backend database object. The vulnerability is not the reference itself, but the "Insecure" aspect: the server fails to verify if the person making the request is actually authorized to access that specific object. Because identifiers are often sequential numbers, attackers can easily iterate through them to recover other customers' details.

Developers frequently attempt to "fix" IDOR vulnerabilities using "Security Through Obscurity," which introduces slightly more complex IDOR variants:

- **Encoded References:** Instead of a plain number, the application might use Base64 encoding (e.g., `Mg==` instead of `2`). An attacker simply needs to recognize the encoding, decode it, increment the value, encode it back, and submit the modified request.
- **Hashed References:** Applications might use cryptographic hashes like MD5 or SHA1 to mask the ID. While this looks like a random string, if an attacker determines the hashing algorithm and the data being hashed (e.g., an MD5 hash of the user ID integer), they can replicate the hashing function to execute the IDOR.
- **Deterministic Generation (UUIDv1):** Some applications use Universally Unique Identifiers (UUIDs). While UUIDv4 is random, UUIDv1 is generated based on the host's MAC address and the exact timestamp of creation. If an attacker knows the exact time window when an object (like a voucher) was generated, they can brute-force the predictable UUID sequence to recover valid codes.

3.4 Remediation (SDOR)

Hiding or obfuscating references does not fix the underlying vulnerability. The only effective remediation is implementing Secure Direct Object Reference (SDOR) by enforcing strict server-side access controls. Whenever a request for an object is received, the backend application must verify that the currently authenticated user's session owns or has explicit permission to interact with the requested object ID.

4. Practical Work

The practical execution phase required intercepting client-side traffic and manipulating local storage data to exploit an authorization bypass on the TryPresentMe application.

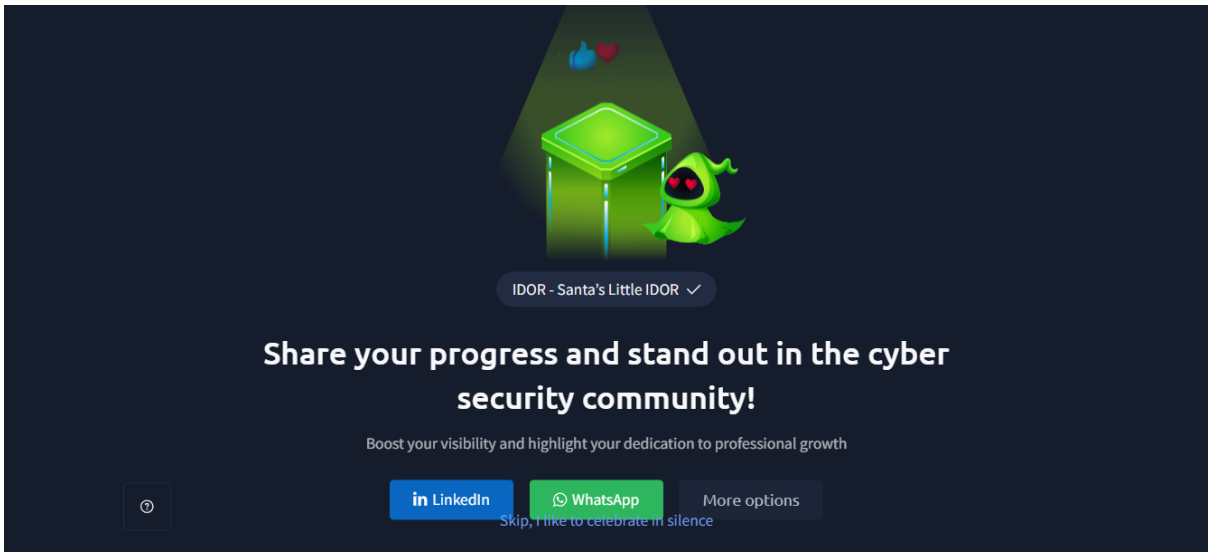
- **Initial Access & Traffic Analysis:**
  - Authenticated to the application using the provided credentials (`niels` / `TryHackMe#2025`).
  - Opened the browser's Developer Tools and navigated to the Network tab to monitor background API requests.
  - Upon refreshing the dashboard, I observed an interesting API call named `view_accountinfo`.
  - Inspecting the request payload revealed that the application was utilizing a `user_id` parameter with a value of `10`, which successfully returned the profile data for the authenticated user.
- **Exploiting the IDOR Vulnerability:**
  - Recognizing that the application was relying on client-supplied data to fetch user records, I navigated to the Storage tab in the Developer Tools.
  - Under the Local Storage dropdown, I located the `auth_user` data entry, which contained the `user_id` key set to `10`.
  - To test for horizontal privilege escalation, I manually modified the `user_id` value from `10` to `11` and refreshed the page.
  - The application failed to validate authorization on the backend, subsequently loading a completely different user's private data.
  - I then iterated through the `user_id` values to fulfill the primary objective. By changing the parameter to `15`, I successfully exploited the IDOR vulnerability to identify the specific parent account that had 10 children assigned to it.

5. Tools Used

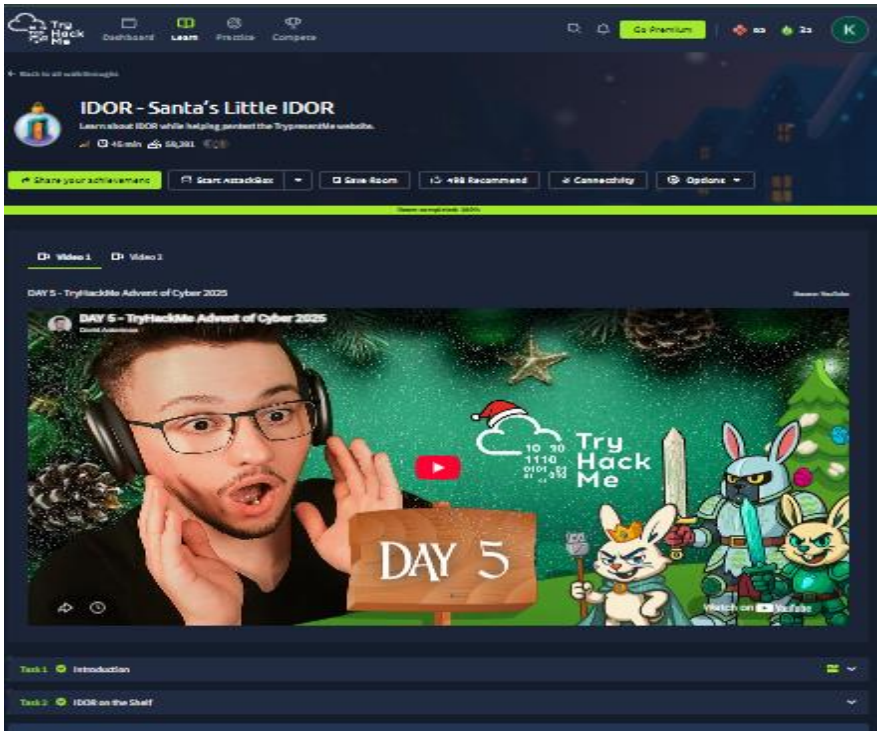
- **Web Browser Developer Tools:** Utilized the Network tab for traffic interception and API analysis, and the Storage tab for manipulating Local Storage parameters.
- **UUID Decoder:** Employed to analyze voucher formats and determine the use of predictable UUID version 1 generation.
- **TryHackMe AttackBox:** Provided the isolated environment to interact with the target machine.

6. Evidence and Screenshots

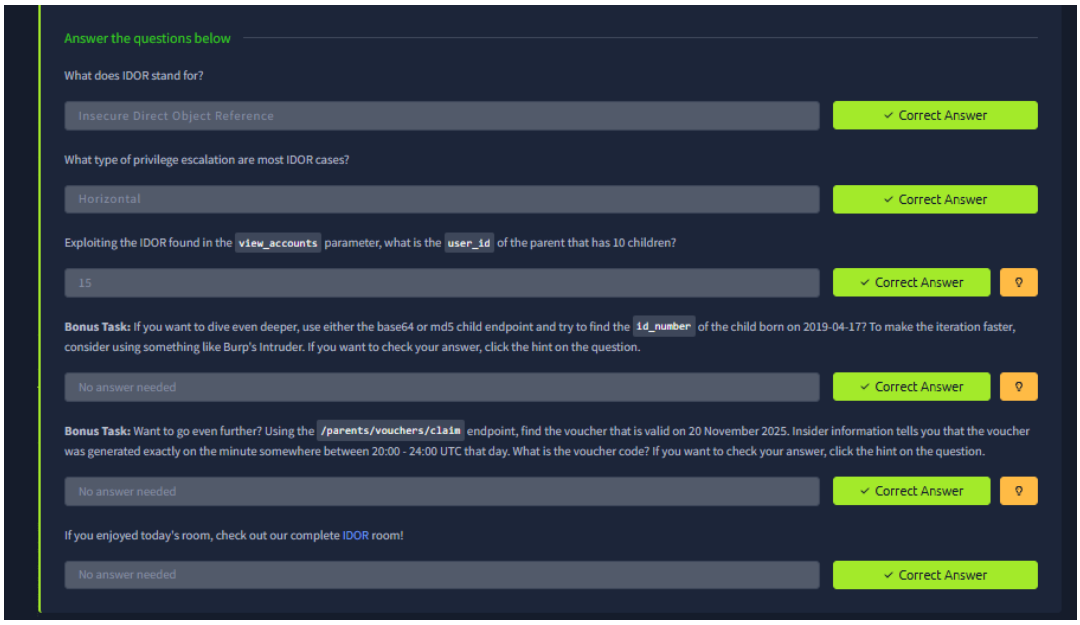
All relevant visual evidence of the completed exercises is stored in the designated submission directory.



- **Fig. 4.1:** TryHackMe IDOR - Santa's Little IDOR room completion certificate (`Complete _2.png`).



- **Fig. 4.2:** Full room completion dashboard for the Advent of Cyber 2025 Day 5 task (`Complete All Task.png`).



- **Fig. 4.3:** Verification of correct answers, confirming IDOR stands for Insecure Direct Object Reference, the vulnerability represents Horizontal privilege escalation, and the successful exploitation revealed `user_id 15` (`complete Questions and Answer .png`).

7. Results

The Access Control and IDOR module was completed successfully. Through traffic analysis and parameter manipulation, I successfully identified that the TryPresentMe application relied exclusively on client-side state (`user_id` in Local Storage) to retrieve sensitive account information. By modifying this value, I successfully executed a horizontal privilege escalation attack, bypassing authorization controls to access unauthorized user records, ultimately identifying user ID 15 as the target.

8. Key Learnings

Completing this vulnerability assessment highlighted critical insights into secure application development:

- **Authentication is Not Authorization:** Just because a user is logged in with a valid session token does not mean they are permitted to view any data in the database. Separate, rigorous authorization logic must be enforced on every object retrieval.
- **Never Trust the Client:** Storing critical identifiers like `user_id` in Local Storage or sending them via easily modifiable API parameters without backend validation is a catastrophic architectural flaw.
- **Obfuscation is a Trap:** Attempting to mitigate IDOR by hashing (MD5) or encoding (Base64) sequential IDs provides a false sense of security. Attackers can easily identify the algorithm, reverse the encoding, or replicate the hash to continue iterating through objects.
- **Predictability is Vulnerable:** Using UUIDv1 for sensitive objects like vouchers allows attackers to predict and generate valid codes if the creation timestamp is known, showcasing the necessity of using cryptographically secure, random identifiers (like UUIDv4) combined with strict access controls.

9. Conclusion

The Santa's Little IDOR practical scenario provided an excellent demonstration of how devastating missing authorization checks can be. While the vulnerability is categorized as Insecure Direct Object Reference, it fundamentally represents a complete failure of the application's access control architecture. By successfully manipulating local storage parameters to view other users' private data and exploring how developers improperly attempt to mask these flaws with encoding and hashes, the absolute necessity of server-side validation was solidified. This understanding is critical for identifying and exploiting logical authorization flaws in the advanced Web Application Penetration Testing tasks.

10. References

1. TryHackMe - IDOR - Santa’s Little IDOR (AoC Day 5)
2. WAPT Phase 1 Task Instructions & Guidelines - IqraSec Academy